

SLURM Quick Reference

Great Lakes / Armis2 | verified 2026-08-20

SUBMIT AND MONITOR

```
sbatch job.sbatch
sbatch --parsable job.sbatch
sbatch --array=1-24%8 array.sbatch
sbatch --dependency=afterok:12345 combine.sbatch

squeue -u "$USER"
squeue -r -j JOB_ID
scontrol show job -dd JOB_ID
scancel JOB_ID
scancel ARRAY_ID_TASK_ID
```

ACCOUNTING

```
seff JOB_ID
seff ARRAY_ID_TASK_ID
sacct -j JOB_ID --units=G \
    -o JobIDRaw,State,Elapsed,TimeLimit,\
    AllocCPUS,ReqMem,MaxRSS,TotalCPU,ExitCode
sstat -j JOB_ID.batch --format=AveCPU,AveRSS,MaxRSS
```

NORMAL R/PYTHON JOB

```
#!/usr/bin/env bash
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --mem=1G
#SBATCH --time=00:05:00
#SBATCH --output=logs/%x_%j.out
#SBATCH --error=logs/%x_%j.err
```

```
set -Eeuo pipefail
cd "${SLURM_SUBMIT_DIR:?}"
module purge
module load R
Rscript analysis.R
```

THREAD LIMITS

```
export OMP_NUM_THREADS="${SLURM_CPUS_PER_TASK:-1}"
export OPENBLAS_NUM_THREADS="$OMP_NUM_THREADS"
export MKL_NUM_THREADS="$OMP_NUM_THREADS"
export NUMEXPR_NUM_THREADS="$OMP_NUM_THREADS"
```

Request more CPUs only when the program actually creates threads or worker processes.

ARRAY ESSENTIALS

```
#SBATCH --output=logs/%x_%A_%a.out
#SBATCH --error=logs/%x_%A_%a.err
TASK_ID="${SLURM_ARRAY_TASK_ID:?}"

sbatch --array=1-24%8 array.sbatch
```

- %A = array job ID; %a = task index.
- %8 = at most eight simultaneously running tasks.
- All elements share the same initial CPU, memory, and time request.
- Use a manifest so task IDs remain stable and auditable.
- Create the log directory before sbatch.

DEPENDENCIES AND RECOVERY

```
sbatch --dependency=afterok:$ARRAY combine.sbatch
sacct -nX -P -j $ARRAY -o JobIDRaw,State
sbatch --array=6,11,19%3 array.sbatch
```

Core rule

Aggregate only after success; rerun failed elements rather than restarting the full analysis.

Resource Tuning and Troubleshooting

Measure first; change one thing at a time

RESOURCE TUNING LOOP

- Run a representative pilot on the same data class and software environment.
- Inspect seff, sacct, exit code, and the application logs.
- Compare TotalCPU with allocated CPU-walltime; inspect MaxRSS across .batch steps.
- Revise CPU, memory, or time and retain a defensible safety margin.
- Validate across multiple tasks; split heterogeneous work into resource classes.
- Scale with a throttled array and inspect the distribution, not one easy task.

INTERPRET SEFF / SACCT

Low CPU, serial code

Request fewer CPUs or parallelize the application.

Low CPU, heavy I/O

Reduce concurrent readers; stage or chunk data.

MaxRSS near request

Add memory margin or reduce peak task size.

OUT_OF_MEMORY

Increase memory or change the algorithm/data chunk.

Elapsed near limit

Increase time, split, or checkpoint.

TIMEOUT

Walltime was insufficient; diagnose stalls before adding CPU.

TROUBLESHOOT IN THIS ORDER

1. State / pending reason

`squeue -o ...%R`; `scontrol show job`. Distinguish Priority, Resources, Dependency, account/QOS limits, and JobArrayTaskLimit.

2. Logs and exit code

Open the element-specific stderr/stdout and read sacct State/ExitCode before changing requests.

3. Resource evidence

Inspect CPU use, MaxRSS, elapsed time, and whether the requested CPUs were configured in the application.

4. Environment and paths

Load modules inside the batch shell; print executable versions; verify working directory, permissions, quotas, and parent directories.

U-M FACTS - VERIFIED 2026-08-20

- Defaults: 60-minute walltime and 768 MB per CPU on both clusters.
- Standard maximum: 14 days; queued-job maximum: 5,000/user/account.
- Login nodes: 2 CPUs and 4 GB. Use `salloc`, `debug`, or `Open OnDemand` for computation.
- Scratch: temporary, not backed up, and subject to 60-day access purge.
- Great Lakes is not approved for HIPAA/PHI; Armis2 requires approved sensitive-data storage and handling.

Resource Right-Sizing

CPU | memory | time | arrays

CPU

- Serial R/Python worker: usually 1 CPU.
- Threaded code: `--cpus-per-task=N` and configure N threads.
- Independent phenotypes/files: prefer array tasks.
- Low CPU can be I/O wait; more CPUs will not fix storage latency.

MEMORY

- Collect MaxRSS across representative tasks and .batch steps.
- Inspect median, upper tail, and outliers.
- Add a safety margin based on variability and failure cost.
- Split outliers into separate resource classes.

TIME

- Walltime is a hard upper limit.
- Use upper-tail elapsed time plus startup and I/O variability.
- Avoid multi-day requests for short jobs.
- Checkpoint or split fragile long-running work.

ARRAYS

- All elements share one initial envelope.
- Throttle with `--array=1-N%K`.
- Use small/medium/large arrays when tasks differ.
- Move to a workflow engine when resources and dependencies multiply.

Evidence-based request

A resource request is a hypothesis. Pilot, inspect, revise, and validate again when sample size, software, or algorithm changes.

U-M billing / scheduling implication

Great Lakes documents charging based on the dominant weighted share of CPU, memory, and GPU. Oversized memory can therefore drive cost and reduce placement flexibility.

SLURM Troubleshooting Decision Tree

Great Lakes / Armis2

1

Did sbatch accept the job?

NO: read the submission error. Check account, partition, directive syntax, limits, path, and execute permission.

2

Accepted but pending?

Use `squeue ... %R` and `scontrol show job`. Interpret Priority, Resources, Dependency, JobArrayTaskLimit, and account/QOS limits.

3

Started and failed?

Use `sacct State/ExitCode` and the element-specific `stderr`. OOM -> memory; TIMEOUT -> walltime; 127 -> environment/command.

4

Completed but inefficient?

Compare TotalCPU with allocated core-walltime; inspect MaxRSS and elapsed distributions; check hidden threads and shared I/O.

5

Logs or results missing?

Create parent directories before `sbatch`; verify WorkDir, `--chdir`, `%A_%a` log names, permissions, quota/inodes, and scratch lifecycle.

DIAGNOSTIC ORDER

STATE / REASON	✓
LOGS	✓
EXIT CODE	✓
RESOURCES	✓
ENVIRONMENT	✓
PATHS / PERMISSIONS	✓
SHARED I/O	

Great Lakes and Armis2 Operational Card

Verified against U-M ARC documentation 2026-08-20

SETTING	GREAT LAKES	ARMIS2
Default walltime	60 minutes	60 minutes
Default memory per CPU	768 MB	768 MB
Standard max walltime	14 days	14 days
Max queued jobs / user / account	5,000	5,000
Login-node envelope	2 CPUs / 4 GB	2 CPUs / 4 GB
Home quota	80 GB/user	80 GB/user
Scratch quota / lifecycle	10 TB + 1M inodes; 60-day access purge; no backup	10 TB; 60-day access purge; no backup
Debug	1 job/user; 4 hours; 8 CPUs; 40 GB	1 job/user; 4 hours; up to 8 CPUs / 40 GB
Sensitive data	HIPAA/PHI not permitted	Sensitive/HIPAA work with approved storage and compliance

Session profile

Single smoke job: debug. Visible parallel array: standard. Start with max concurrency 8 or lower. Pass the account via the submission wrapper rather than hard-coding it.

Storage and compliance

Use scratch for active intermediates, not archival storage. Keep Great Lakes free of PHI. On Armis2, use only approved sensitive-data storage and handling.